

Unit 2 Fast Track

Classes, object lifetime and dynamic objects

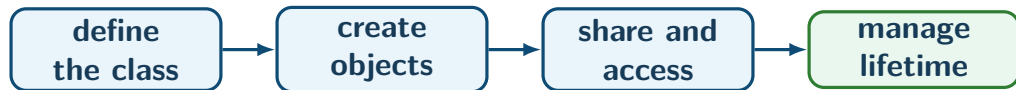
Aditya Tandon

25CS301 – Object Oriented Programming System

Lectures 6–8

Members ■ Constructors ■ Friends ■ Dynamic Objects

Unit 2 follows the complete life of an object



- Class members and data hiding
- Constructors, destructors and static members
- Friend functions, objects as arguments and `this`
- Arrays, nested classes, pointers and smart pointers

Remember: The unit is one story: define an object, initialize it, use it, and release it safely.

Use dot for an object and arrow for an object pointer

```
1 class Student {  
2 public:  
3     int roll;  
4     void show() const;  
5 };  
6  
7 Student s;  
8 s.roll = 101;  
9 s.show();           // object: dot
```

```
1 Student* p = &s;  
2 p->roll = 102;  
3 p->show();           // pointer: arrow
```

Two access operators

object.member

pointer->member

Member functions may be defined inside or outside the class

Inside the class

```
1 class Student {  
2     private:  
3         int roll;  
4     public:  
5         int getRoll() const { return roll; }  
6 };
```

Outside the class

```
1 class Student {  
2     private:  
3         int roll;  
4     public:  
5         int getRoll() const;  
6 };  
7  
8 inline int Student::getRoll() const  
9 { return roll; }
```

Remember: `Student::` tells the compiler that the function belongs to `Student`.

Data hiding keeps object state under class control

```
1 class BankAccount {  
2     public:  
3         void deposit(double amount) {  
4             if (amount > 0)  
5                 balance += amount;  
6         }  
7  
8         double getBalance() const {  
9             return balance;  
10        }  
11    private:  
12        double balance = 0;  
13};
```

Why private data?

- invalid changes can be rejected;
- class rules stay in one place;
- internal representation may change later.

Use const when a function should only read an object

```
1 class Student {  
2 public:  
3     void display() const;           // does not change this object  
4 };  
5  
6 void printStudent(const Student& s) {  
7     s.display();                   // no copy; no modification  
8 }
```

const Student&

The parameter refers to the original object but cannot modify it.

display() const

The member function promises not to modify the object's visible state.

Objects can be passed to functions like other values

```
1 class Rectangle {  
2 public:  
3     Rectangle(double w, double h) : width{w}, height{h} {}  
4     double area() const { return width * height; }  
5 private:  
6     double width, height;  
7 };  
8  
9 void compare(const Rectangle& a, const Rectangle& b) {  
10     cout << (a.area() > b.area() ? "A" : "B");  
11 }
```

Remember: Prefer a const reference when the function only needs to read a large object.

A function may also return a complete object

```
1 class Complex {
2 public:
3     Complex(double r, double i) : real{r}, imag{i} {}
4     double realPart() const { return real; }
5     double imagPart() const { return imag; }
6 private:
7     double real, imag;
8 };
9
10 Complex add(const Complex& a, const Complex& b) {
11     return Complex{
12         a.realPart() + b.realPart(),
13         a.imagPart() + b.imagPart()
14     };
15 }
```

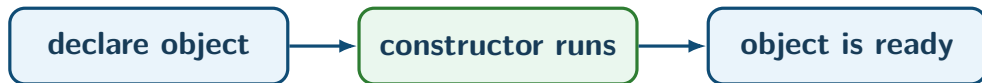

A constructor gives every object a starting state

Without a constructor

The programmer must remember to initialize every member after creating every object.

With a constructor

Initialization happens automatically when the object is created.



A constructor has the same name as the class

```
1 class Student {  
2 public:  
3     Student() {  
4         cout << "Student created";  
5     }  
6 };  
7  
8 int main() {  
9     Student s1;    // constructor runs automatically  
10 }
```

- same name as the class;
- no return type;
- called during object creation.

A default constructor needs no arguments

```
1 class Student {  
2 public:  
3     Student() : roll{0}, name{"Not assigned"} {}  
4 private:  
5     int roll;  
6     string name;  
7 };  
8  
9 Student s1;           // Student()
```

Compiler-provided form

If a class declares no constructor, the compiler may provide a default constructor. Built-in members still need suitable default values.

A parameterized constructor receives starting values

```
1 class Student {  
2 public:  
3     Student(int r, string n)  
4         : roll{r}, name{n} {}  
5 private:  
6     int roll;  
7     string name;  
8 };  
9  
10 Student s1{101, "Aman"};
```

Remember: The initializer list places values directly into the data members.

Constructor overloading supports different ways to create an object

```
1 class Time {  
2 public:  
3     Time() : hour{0}, minute{0} {}  
4     Time(int h) : hour{h}, minute{0} {}  
5     Time(int h, int m) : hour{h}, minute{m} {}  
6 private:  
7     int hour, minute;  
8 };  
9  
10 Time t1;  
11 Time t2{9};  
12 Time t3{9, 30};
```

A copy constructor creates a new object from an existing object

```
1 class Student {  
2 public:  
3     Student(int r) : roll{r} {}  
4     Student(const Student& other)  
5         : roll{other.roll} {}  
6 private:  
7     int roll;  
8 };  
9  
10 Student first{101};  
11 Student second{first};
```

Remember: The usual parameter is `const ClassName&`.

A constructor may allocate storage needed by the object

```
1 class List {  
2 public:  
3     explicit List(int n)  
4         : size{n}, data{new int[n]} {}  
5  
6     ~List() { delete[] data; }  
7 private:  
8     int size;  
9     int* data;  
10 };
```

Traditional term

Some textbooks call this a dynamic constructor because memory is allocated at run time.

Modern preference

Use vector or a smart pointer when possible.

The constructor runs once for every object created

```
1 Student s1;  
2 Student s2;  
3 Student group[3];
```

- s1: one constructor call
- s2: one constructor call
- group[3]: three constructor calls

Total

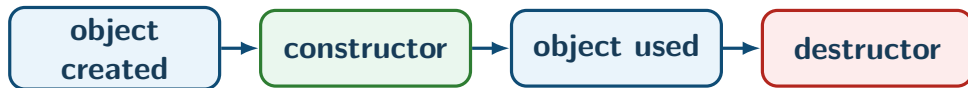
Five objects produce five constructor calls.

A destructor performs cleanup when an object is destroyed

```
1 class FileLog {  
2 public:  
3     FileLog() {  
4         cout << "Log opened\n";  
5     }  
6  
7     ~FileLog() {  
8         cout << "Log closed\n";  
9     }  
10 };
```

- name begins with the tilde symbol ~;
- has no return type and no parameters;
- only one destructor exists in a class.

Object lifetime has a simple beginning and end



Scope controls automatic objects

A local object's constructor runs when execution reaches its declaration. Its destructor runs when execution leaves that scope.

Remember: Constructor starts the usable lifetime; destructor closes it.

A const object can call only const member functions

```
1 class Circle {  
2     public:  
3         explicit Circle(double r) : radius{r} {}  
4         double area() const {  
5             return 3.14 * radius * radius;  
6         }  
7     private:  
8         double radius;  
9 };  
10  
11 const Circle c{2.0};  
12 cout << c.area();
```

Remember: The constructor initializes the object first; const restrictions apply after initialization.

A static member belongs to the class, not one object

```
1 class Student {  
2     public:  
3         static string college;  
4         static void showCollege() {  
5             cout << college;  
6         }  
7     };  
8  
9     string Student::college = "ABC Institute";  
10  
11     Student::showCollege();
```

- one shared copy of a static data member;
- static member functions are normally called with the class name.

Constructors and destructors can maintain a shared object count

```
1 class Student {  
2 public:  
3     Student() { ++count; }  
4     Student(const Student&) { ++count; }  
5     ~Student() { --count; }  
6     static int getCount() { return count; }  
7 private:  
8     inline static int count = 0;  
9 };
```

create object
count increases



destroy object
count decreases

A friend function receives controlled access to private data

```
1 class Box {  
2     friend double volume(const Box&);  
3 public:  
4     Box(double l, double w, double h)  
5         : length{l}, width{w}, height{h} {}  
6 private:  
7     double length, width, height;  
8 };  
9  
10 double volume(const Box& b) {  
11     return b.length * b.width * b.height;  
12 }
```

Remember: A friend is not a member function, but the class explicitly grants it access.

A friend class grants access to another class

```
1 class Inspector;
2
3 class Machine {
4     friend class Inspector;
5 private:
6     int temperature = 75;
7 };
8
9 class Inspector {
10 public:
11     void check(const Machine& m)
12     { cout << m.temperature; }
13 };
```

Use carefully

Friendship increases coupling. Grant it only when two classes genuinely need close cooperation.

The this pointer refers to the current object

```
1 class Student {  
2     public:  
3         Student(int roll) {  
4             this->roll = roll;  
5         }  
6  
7         Student& setRoll(int roll) {  
8             this->roll = roll;  
9             return *this;  
10        }  
11    private:  
12        int roll;  
13    };  
14  
15    Student s{101};  
16    s.setRoll(102);
```


An array stores several objects of the same class

```
1 class Student {  
2 public:  
3     Student() : roll{0} {}  
4     void setRoll(int r) { roll = r; }  
5     void show() const { cout << roll << '\n'; }  
6 private:  
7     int roll;  
8 };  
9  
10 Student group[3];  
11  
12 for (int i = 0; i < 3; ++i) {  
13     group[i].setRoll(101 + i);  
14     group[i].show();  
15 }
```

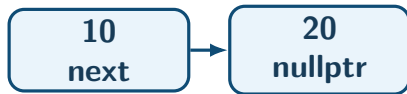
A nested class groups a helper type inside its owner

```
1 class University {  
2 public:  
3     class Department {  
4     public:  
5         void show() const {  
6             cout << "Computer Science";  
7         }  
8     };  
9 };  
10  
11 University::Department d;  
12 d.show();
```

Remember: Use a nested class when the helper type is meaningful mainly inside the outer class.

A self-referential class stores a pointer to the same class type

```
1 class Node {  
2 public:  
3     Node(int v) : value{v}, next{nullptr} {}  
4     int value;  
5     Node* next;  
6 };  
7  
8 Node first{10};  
9 Node second{20};  
10 first.next = &second;
```



Used in

linked lists, trees and graphs

A pointer can store the address of an existing object

```
1 class Student {  
2 public:  
3     void show() const {  
4         cout << "Student";  
5     }  
6 };  
7  
8 Student s;  
9 Student* ptr = &s;  
10  
11 ptr->show();  
12 (*ptr).show();    // equivalent form
```

Remember: Use `->` when accessing a member through a pointer.

new creates a dynamic object; delete destroys it

```
1 Student* ptr = new Student{101, "Aman"};  
2  
3 ptr->show();  
4  
5 delete ptr;           // destructor runs  
6 ptr = nullptr;
```

Main risk

Forgetting delete causes a memory leak. Using a pointer after deletion causes undefined behavior.

Remember: Learn new/delete, but prefer smart pointers in new code.

nullptr clearly represents no object

```
1 Student* ptr = nullptr;  
2  
3 if (ptr != nullptr) {  
4     ptr->show();  
5 }
```

Preferred

nullptr has a dedicated pointer-related type and works safely with overloads.

Avoid in modern C++

Do not use 0 or NULL when nullptr expresses the intent directly.

unique_ptr represents one clear owner

```
1 #include <memory>
2
3 auto ptr = make_unique<Student>(101, "Aman");
4 ptr->show();
5
6 // no delete: object is destroyed automatically
```

- only one unique_ptr owns the object;
- ownership can be moved, not copied;
- cleanup is automatic.

Remember: Use unique_ptr for dynamic objects with one owner.

shared_ptr allows several owners when sharing is necessary

```
1 #include <memory>
2
3 auto first = make_shared<Student>(101, "Aman");
4 auto second = first;
5
6 first->show();
7 second->show();
```

- both pointers refer to the same object;
- a reference count tracks the owners;
- the object is destroyed when the final owner disappears.

Choose the simplest ownership model that fits

Tool	Meaning	Typical use
Ordinary object	Automatic lifetime	Default choice for local values
Raw pointer	Observes an object	Non-owning access; legacy interfaces
<code>unique_ptr</code>	One owner	Dynamic object with exclusive ownership
<code>shared_ptr</code>	Several owners	Truly shared lifetime
<code>nullptr</code>	No object	Safe empty pointer state

Remember: Do not use shared ownership when a normal object or one owner is enough.

One small class can combine the main Unit 2 ideas

```
1 class Student {  
2     int roll;  
3     string name;  
4     inline static int count = 0;  
5 public:  
6     Student(int r, string n)  
7         : roll{r}, name{n} { ++count; }  
8     ~Student() { --count; }  
9     void show() const {  
10         cout << roll << " " << name;  
11     }  
12     static int total() { return count; }  
13 };
```

Concepts combined

- data hiding
- parameterized constructor
- initializer list
- destructor
- const member function
- static member

Unit 2 can be remembered in six lines

- ① Keep data private and expose meaningful member functions.
- ② Pass read-only objects using const references.
- ③ Use constructors to initialize and destructors to clean up.
- ④ Use static members for class-wide information.
- ⑤ Use friends only for necessary cooperation.
- ⑥ Prefer automatic objects and smart pointers over manual memory management.

Remember: A good class controls its data, begins valid, and manages its lifetime clearly.

Assignment: apply the complete Unit 2 toolkit

- ❶ Create a **LibraryBook** class with private data, default and parameterized constructors, a destructor and const display function.
- ❷ Pass two books to a function and return the book with the higher price.
- ❸ Maintain a static count of live book objects.
- ❹ Create an array of three books and one dynamic book using `unique_ptr`.
- ❺ Write five lines explaining where `this`, `nullptr`, a friend function or a nested class could be useful.

Next: Unit 3 – Polymorphism and Operator Overloading